



LAB 4

x86-64 Assembly: Assemble!

Buffer Slide

Discussing doubts/solutions attempts for last week's inlab / labexam / outlab

All previous labexam solutions are uploaded on the repo



Alert Slide

Please meet the TAs if you received **“Invalid Format”** or **similar comment** in a previous inlab



This you?

Why Assembly?

- **1:1 Mapping:** Assembly is the last step before raw machine code; one line $\approx$ one CPU instruction with a direct bijection.
- **Zero Abstractions:** There are no garbage collectors, heavy runtimes, or safety nets.
- **Hardware Control:** You directly control each and every register and byte.
- **Maximum Speed:** With proper assembly, you can beat compilers in execution speed.

With great power (of the CPU) comes great responsibility (for every single bit).

```
section .text
global _start

_start:
    mov rax, 60 ; sys_exit
    xor rdi, rdi ; status: 0
    syscall

; Machine Code (12 bytes):
48 C7 C0 3C000000 48 31 FF 0F 05
```

task0/exit.s

Godbolt it

The screenshot displays the Godbolt Compiler Explorer interface. On the left, the C++ source code is shown, featuring a `Point` struct and a `distance_sq` function. On the right, the generated x86-64 assembly for the same code is displayed. The interface includes a top bar with navigation options and a bottom bar with compiler details. Several elements are highlighted with yellow boxes: the '+' icon in the toolbar, the 'C++' language dropdown, the 'x86-64 clang 22.1.0' compiler dropdown, the '-std=c++23 -O0 -g' flags dropdown, and the 'Output (/0)' button at the bottom.

```
#include <iostream>

struct Point {
    int x;
    int y;
};

int distance_sq(Point p) {
    return p.x * p.x + p.y * p.y;
}

int main() {
    Point p(3, 4);
    std::cout << distance_sq(p) << '\n';
}
```

```
distance_sq(Point):
    push    rbp
    mov     rbp, rsp
    mov     qword ptr [rbp - 8], rdi
    mov     eax, dword ptr [rbp - 8]
    imul    eax, dword ptr [rbp - 8]
    mov     ecx, dword ptr [rbp - 4]
    imul    ecx, dword ptr [rbp - 4]
    add     eax, ecx
    pop     rbp
    ret

main:
    push    rbp
    mov     rbp, rsp
    sub     rsp, 16
    mov     rax, qword ptr [rip + .L__const.main.p]
    mov     qword ptr [rbp - 8], rax
    mov     rax, qword ptr [rbp - 8]
    mov     qword ptr [rbp - 16], rax
    mov     rdi, qword ptr [rbp - 16]
    call    distance_sq(Point)
    mov     esi, eax
    mov     rdi, qword ptr [rip + std::cout@GOTPCREL]
    call    std::ostream::operator<<(int)@PLT
    mov     rdi, rax
    mov     esi, 10
    call    std::basic_ostream<char, std::char_traits<char>>& std::operator<<
    xor     eax, eax
    add     rsp, 16
    pop     rbp
    ret

.L__const.main.p:
    .long   3
    .long   4
```

Compiler Explorer Godbolt

- Use godbolt.org to see what your C/C++ code compiles to
- `clang` with `-O0 -g` gives nice readable code
- Click on **Output** at the bottom to see compiler stderr/stdout
- Click on some C/C++ code and press **Ctrl+F10** to see the corresponding assembly
- Click on the **+** button to add more tabs
- You can explore various compilers and see what the compiled output looks like

The screenshot displays the Compiler Explorer Godbolt interface with three tabs open:

- C++ source #1**:

```
1 int main() {  
2     int x = 7;  
3     int y = x + 10;  
4     return y;  
5 }
```
- x86-64 clang 22.1.0 (Editor #1)**:

```
1 main:  
2     push    rbp  
3     mov     rbp, rsp  
4     mov     dword ptr [rbp - 4],  
5     mov     dword ptr [rbp - 8],  
6     mov     eax, dword ptr [rbp -  
7     add     eax, 10  
8     mov     dword ptr [rbp - 12],  
9     mov     eax, dword ptr [rbp -  
10    pop     rbp  
11    ret
```
- mips clang 22.1.0 (Editor #1)**:

```
1 main:  
2 $func_begin0 = $tmp0  
3     addiu   $sp, $sp, -24  
4     sw      $ra, 20($sp)  
5     sw      $fp, 16($sp)  
6     move    $fp, $sp  
7     addiu   $1, $zero, 0  
8     sw      $zero, 12($fp)  
9     addiu   $1, $zero, 7  
10    sw      $1, 8($fp)  
11    lw       $1, 8($fp)  
12    addiu   $2, $zero, 10
```

Sections in Assembly

Sections group data and instructions based on their access permissions, initialization requirements etc.

- **section .data** initialized storage: constants, arrays, buffers that exist before the program runs.
- **section .text** your actual instructions live here, organized under labels.
- **global** makes a symbol visible to the linker, allowing external object files or the runtime environment to reference it.
- **_start** is a special label marking the default entry point where program execution begins.

```
section .data
    a: dq 10
    b: dq 20

section .text
    global _start

_start:
    ; assemble here
```

task0/add.s

Labels in Assembly

- **Labels** act as named markers in your code, representing memory addresses of instructions or data.
- They end with a colon (:) when defined, serving as targets for jumps, loops, and function calls.
- **jmp label** redirects execution flow straight to the labeled instruction.
- **Recall from theory:** How do you track which instruction is being executed? What does a “jump” mean then?
- Can a label be used for a non-jump instruction?

```
section .text
global _start

_start:
; main execution point
mov rcx, 5

.loop_start:
dec rcx
jnz .loop_start; jump if != 0
```


Registers: Big Potato Stuff

- **Registers** are tiny, blazing-fast storage slots built directly into the CPU taking almost zero time to access.
- **16 General-Purpose Registers**: rax, rbx, rcx, rdx, rsi, rdi, rbp, rsp, r8–r15.
- The same GPR can be accessed in many bit-widths
- rax → eax → ax → (ah, al); rdi → edi → di → dil; r8 → r8d → r8w → r8b
- **Specialized Registers** also exist for floating-point/vector operations, status etc (such as **xmm0–xmm15** for 128-bit SIMD, **ymm0–ymm15** for 256-bit SIMD, **eflags**, the status register and many more).

One Register, Several Widths (Nesting of **rax**)



Basic Instructions

mov dst, src

copy a value into a **register** or **memory location**

add/sub dst, value

add / subtract, in place

imul dst, value

signed multiply

xor reg, reg

the idiomatic way to zero a register

inc/dec dst

increment / decrement by 1

cmp a, b

compute $a - b$, discard result, but update some flags

push/pop value

save / restore a value on the **stack**

lea dst, addr

compute a memory address without loading it



What is a value? Reg? Memory? Constant/Immediate value? What is addr? – **Coming Later**

Instruction Operands & Inputs

Assembly instructions perform operations on data, but where does that data come from?

The CPU handles three distinct types of **operands** (inputs and destinations) depending on where the values are physically stored and how quickly they need to be accessed.

Registers (reg)

Blazing-fast storage locations built directly inside the CPU core. Limited in number but near-instant to access.

```
mov rdi, rax
mov dst_reg, src_reg
```

Memory (*mem)

System RAM addresses, written inside brackets. Accessing memory is much slower than registers.

```
mov rax, [rbx]
mov reg, [mem_addr]
```

Immediates (imm)

Hardcoded constant values or raw numbers embedded directly within the instruction itself.

```
mov rbx, 42
mov reg, constant
```

Crucial Hardware Constraint: You **cannot** transfer data directly from memory to memory (e.g., `mov [rsi], [rdi]` is illegal).

Can you taste your code?

- Exact assembly syntax – order of arguments, what operands are allowed, some fancy symbols (\$,%) – depends on the **syntax flavour**

- For eg: to move value in `rbx` to `rax` ⇒

Intel: `mov rax, rbx`

AT&T: `movq %rbx, %rax`

- In AT&T, some instructions (like `mov`) have suffixes that tell the CPU what operand size to use or how to do bit extension
- E.g. `b/w/l/q` specify size; `z` indicates zero-extension, as opposed to sign-extension
- How to view disassembled output from a compiled file?
`objdump -d -M intel file.o` → To see Intel syntax
`objdump -d -M att file.o` → To see AT&T syntax
- Or use godbolt.org :)



Assembly follows my brother

```
static long a = 10;  
static long b = 20;
```

```
long f(long a, long b) {  
    return a + b;  
}
```

```
int main() {  
    return f(a, b);  
}
```



```
section .data  
    a: dq 10  
    b: dq 20
```

```
section .text  
global _start
```

```
f:  
    mov rax, rdi  
    add rax, rsi  
    ret
```

```
_start:  
    mov rdi, [a]  
    mov rsi, [b]  
    call f  
  
    mov rdi, rax  
    mov rax, 60  
    syscall
```

Mommy where's my data?

```
section .data
    a:      db 10          ; 1 byte
    b:      dw 1000        ; 2 bytes
    c:      dd 100000      ; 4 bytes
    d:      dq 10000000000 ; 8 bytes
    arr:     dq 1, 2, 3, 4 ; multiple values
    msg:     db "Hello", 0 ; string

section .bss
    buffer: resb 64        ; reserve 64 bytes
    arr2:    resq 10       ; reserve 10 × 8 bytes
```

- `.data` = initialized data
- `.bss` = uninitialized storage
- `db/dw/dd/dq` define 1/2/4/8-byte values, while `resb/resw/resd/resq` reserve 1/2/4/8-byte spaces
- label (eg: `a`, `b`, `arr`, `msg`) allows you to access these in code

From Source Code to Executable

1. Source (.c) -> Assembly (.s)

```
$ gcc -S hello.c -o hello.s
```

2. Assembly (.s) -> Obj File (.o)

```
$ nasm -f elf64 hello.s -o hello.o
```

-f specifies format (using ELF64 here)

(Or, `objdump -d -M intel hello.o`)

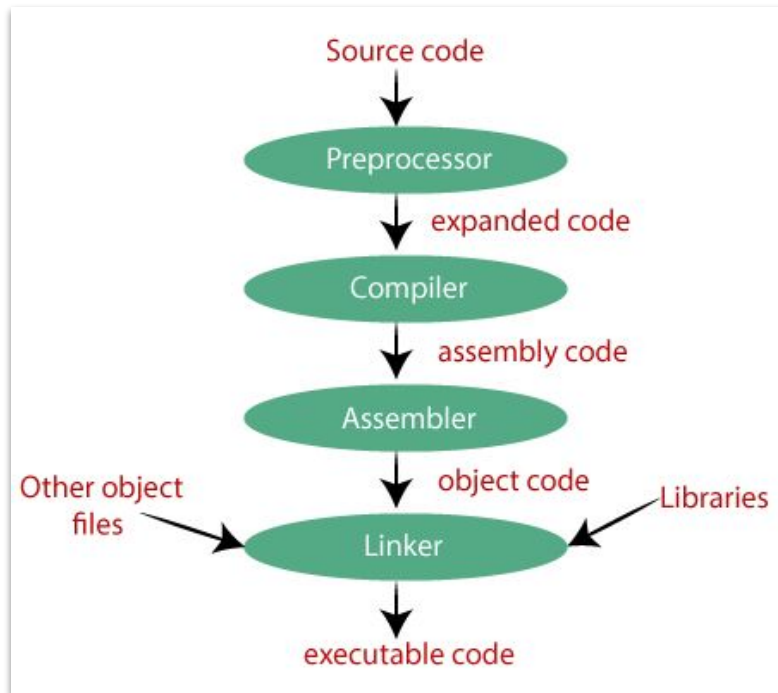
3. Obj file (.o) -> Executable (.out or none)

```
$ ld hello.o main.o -o main
```

- You can use **libc (standard C library)** (eg. printf etc.) in assembly by linking with it!

- Easy: Use GCC: `gcc hello.s -o hello`

- Or: Use ld: `ld hello.o -o hello -lc -dynamic-linker /lib64/ld-linux-x86-64.so.2`



Avengers Assemble (tinyy potato) – tasko!



`add.s – move a into rdi, add b, exit with the result`

Jumps and Branching

- **cmp a, b** sets CPU flags based on $(a - b)$. Subsequent jump instructions can use these flags to jump conditionally
- **jmp label** — unconditional, always taken.
- **je / jne** — jump if equal / not equal.
- **j1 / jle / jg / jge** — signed $</\le/ >/\ge$.
- **jb / jbe / ja / jae** — unsigned $</\le/ >/\ge$.
- There's no if/else keyword — you build control flow yourself, one cmp and one jump at a time.
- An “if” in assembly is a comparison, a conditional jump past the body, and a label to land on.

```
jnc .else ; opposite of cond
.if:
; code to run if cond is true
jmp .end
.else:
; code to run when cond is false
.end:
```

task1 (medium potato): Rectangles!



`overlap.s` – do `rect_a` and `rect_b` overlap? 1 if yes, 0 if no

Arrays

- An array is just consecutive values in **memory**
- `dq 5, -2, 8, 3, -1, 6` defines six 64-bit values, back to back.
- `[vec1 + 8*rcx]` reads index `rcx`; we have to multiply `rcx` by 8 as the value is in bytes and each element is 8 bytes long here.
- Get the stride wrong and you silently read wrong bytes.
- Can 8 be replaced by any arbitrary integer?

```
vec1:
    dq 5, -2, 8, 3, -1, 6
.end:
vecsize equ (vec1.end - vec1)/8

    mov rsi, [vec1+8*rcx]
```

Memory Access Modes

$[base + index * scale + displacement]$

- In x86-64 assembly, the processor calculates address offsets in hardware using a single, unified instruction formula: $[base + index * scale + displacement]$.
- This powerful addressing mode lets you locate elements in arrays, structs, or matrices in a single instruction cycle without manual shift arithmetic.

Base [reg1]

Any 64-bit general-purpose register pointing to the start of the structure or memory block.

Index [reg2]

A register holding the dynamic index (e.g., loop counter) of the element to be accessed.

Scale [* 2^x]

A hardware multiplier: can only be 1, 2, 4, or 8 to match standard element widths (byte, word, dword, qword).

Displacement [+ C]

A constant immediate value (offset) added to the address. Ideal for accessing a specific field within an array of structures or skipping headers.

- Eg: `mov eax, [rbx + rcx*4 + 8]`
- Equivalent to: `eax = *(int *) (rbx + rcx*4 + 8)`

Loops

- **No `for/while` keyword.** A loop in assembly is just four ingredients: a **label** to jump back to, a **bound check**, a **body**, and a **jump back to the check**. You assemble these yourself, every time.
- **The four-step pattern:**
 - **1. Init:** set up your index/accumulator with your starting values
 - **2. Check:** compare against bound (repeating condition)
 - **3. Body:** do the actual work for the iteration.
 - **4. Advance:** update the index, then jump back to re-check.
- **Where you check matters.** Both of the versions to the right do the same thing, but **removing the top check** in the upper version makes it a `do {} while()` loop

```
xor rcx, rcx ; i = 0
cmp rcx, N
jge .forend
.forbegin:
; ...
; loop body
; ...
inc rcx
cmp rcx, N
jl .forbegin
.forend:
```

```
xor rcx, rcx ; i = 0
.forbegin:
cmp rcx, N
jge .forend
; ...
; loop body
; ...
inc rcx
jmp .forbegin
.forend:
```

Assemble task2: Dot Product!



`dotprod.s` – loop, multiply, accumulate, exit with the result

Who's the Impostor?

- Which for loop is correct/better to use?

```
.for:
    mov rsi, qword [vec1+8*rcx]
    imul rsi, qword [vec2+8*rcx]
    add rdi, rsi
    inc rcx
    cmp rcx, vecsize
    jl .for
```

```
.for:
    cmp rcx, vecsize
    jge .done

    mov rsi, [vec1 + 8*rcx]
    imul rsi, [vec2 + 8*rcx]
    add rdi, rsi

    inc rcx
    jmp .for
```

It's Doomin' Time

Food for thought:

1. x86 provides various operand combinations, as opposed to MIPS32 which mostly uses register–register operations. Do you think this makes x86 code faster, more convenient, both, or neither?
2. If registers are much faster to access than memory, why do CPUs use a relatively small number of registers instead of thousands of them?
3. `mov` is one of the most fundamental instructions in assembly. Do you think x86's `mov` alone is sufficient to express any computation?
4. (Explore) Are there x86 instructions that can do an arbitrary amount of “work”, as opposed to MIPS32 instructions which perform a fixed amount of work? What might be an example?

It's graded lab time

Starting at 4:05.. Ending at ..



1. Log in to:
Username: **CS231-labexam**
Password: doomsday
2. Extract labexam tarball and work in the extracted folder
3. Go into the same folder and run
`./build-archive.sh`
4. Log in to moodle and submit into the **correct activity: "Inlab Week 4 Assessment"**